

PART B: Cryptography

Question 1

a) Code Screenshots:

Code Snippets

This snippet imports the readline module, which enables the program to read user input from the terminal.

```
// To take the user input import the readline module
const readline = require('readline');
```

A capital letter (A–Z) can be changed to a number between 0 and 25 using this function. For example, A turns into 0, B into 1,..., Z into 25.

```
// Assign letter to specific number
function letter_to_number(letter) {

    // Get the ASCII for A
    a = 'A'.charCodeAt(0);

    // Get the ASCII for the input letter
    b = letter.charCodeAt(0);

    // Calculate the difference to get 0 to 25 range
    calc = b - a
    return calc;
}
```

A number between 0 and 25 is converted back to a capital letter (A–Z) by this function. For example, 0 turns into A, 1 into B, and 25 into Z.

```
// Convert number (0 to 25) back to uppercase letter
function number_to_letter(number) {
    // Get the ASCII code for 'A'
    a = 'A'.charCodeAt(0);

    // Add number to ASCII of 'A' to get target letter
    c = (number % 26) + a;

    // Convert ASCII back to letter
    return String.fromCharCode(c);
}
```

The Encryption function uses modulo 26 arithmetic to shift each letter according to its corresponding letter in the key, thereby encrypting the plaintext. The resulting string of encrypted letters is the ciphertext.

```
// Encryption function
function Encryption(plaintext, key) {

    // Get the key by making the length same as plaintext
    key = key.substring(0, plaintext.length);

    // Empty string to store the cipher text
    let cipher = '';

    // Loop through each letter in the plaintext
    for (let i = 0; i < plaintext.length; i++) {

        // Convert letter to number, Add plaintext and key numbers, then mod 26 to get the remainder.
        // Convert result back to letter and add to cipher
        cipher += number_to_letter((letter_to_number(plaintext[i]) + letter_to_number(key[i])) % 26);
    }

    // Final cipher text
    return cipher;
}
```

After repeatedly requesting that the user enter valid uppercase plaintext, the prompt function verifies the input. It also asks the user to input a valid key. It displays the results and encrypts the text after receiving valid input, then exits the input interface.

```
// Prompt function
function prompt() {
    // Provide a user interface for writing output and reading input
    const rl = readline.createInterface({
        input: process.stdin, // input from keyboard
        output: process.stdout // output on screen
    });

    // Question to ask the user for input
    rl.question('Enter the plaintext: ', (plaintext) => {

        // If the user key in lowercase, number, or symbols
        if (!/[A-Z]/.test(plaintext)) {
            // Error message for user
            console.log("Error: No lowercase, numbers, or symbols allowed.\n");

            // Close after printing the console log statement to ensure the output is not doubled or tripled
            rl.close();

            // Repeat the function to prompt the user again
            prompt();

            return;
        }

        // Nested function to ask the key again if it is invalid
        function Key_again() {
            // Question to ask the user to input the key
            // THIS IS AN EXAMPLE KEY IN COMPUTER SECURITY EXAM
            rl.question('Enter the key (uppercase letters only): ', (key) => {
                // If the user key in lowercase, number, or symbols
                if (!/[A-Z]/.test(key)) {
                    // Error message for user
                    console.log("Error: No lowercase, numbers, or symbols allowed in key.\n");

                    // Recursive function to ask the user to input the key again until the valid one appears
                    Key_again();

                    return;
                }

                // Encrypt the user input
                const ciphertext = Encryption(plaintext, key);

                // Output console log statements
                console.log("-----")
                console.log("      Output")
                console.log("-----")
                console.log("Plaintext: ", plaintext); // Plaintext
                console.log("Key:      ", key.substring(0, plaintext.length)); // Key
                console.log("Cipher text: ", ciphertext); // cipher text
                console.log("\n")

                // Detailed output console log statements
                console.log("-----")
                console.log("Detailed Output")
                console.log("-----")
                // Print letter with number for plain text
                console.log("Plaintext: ", plaintext.split('').map(c => `${c} (${letter_to_number(c)})`).join(' '));
                // Print letter with number for key
                console.log("Key:      ", key.substring(0, plaintext.length).split('').map(c => `${c} (${letter_to_number(c)})`).join(' '));
                // Print letter with number for cipher text
                console.log("Cipher text: ", ciphertext.split('').map(c => `${c} (${letter_to_number(c)})`).join(' '));

                rl.close();
            });
        }

        Key_again();
    });
}
```

INPUT/OUTPUT

Final Output & Detailed Output with its assigned numbers:

```
Enter the plaintext: SUBATHRA
Enter the key (uppercase letters only): THISISANEXAMPLEKEYINCOMPUTERSECURITYEXAM
-----
                        Output
-----
Plaintext:  SUBATHRA
Key:       THISISAN
Cipher text: LBJSBZRN

-----
                        Detailed Output
-----
Plaintext:  S (18) U (20) B (1) A (0) T (19) H (7) R (17) A (0)
Key:       T (19) H (7) I (8) S (18) I (8) S (18) A (0) N (13)
Cipher text: L (11) B (1) J (9) S (18) B (1) Z (25) R (17) N (13)
```

Error Output if the user key in lowercase, numbers or symbols as their input for plaintext:

```
Enter the plaintext: Subathra1234#@
Error: No lowercase, numbers, or symbols allowed.

Enter the plaintext: █
```

Error Output if the user key in lowercase, numbers or symbols as their input for key:

```
Enter the plaintext: SUBATHRA
Enter the key (uppercase letters only): THISisAn1234@#
Error: No lowercase, numbers, or symbols allowed in key.

Enter the key (uppercase letters only): █
```

Final Code

```
1  // To take the user input import the readline module
2  const readline = require('readline');
3
4  // Assign letter to specific number
5  function letter_to_number(letter) {
6      // Get the ASCII for A
7      a = 'A'.charCodeAt(0);
8
9      // Get the ASCII for the input letter
10     b = letter.charCodeAt(0);
11
12     // Calculate the difference to get 0 to 25 range
13     calc = b - a
14     return calc;
15 }
16
17 // Convert number (0 to 25) back to uppercase letter
18 function number_to_letter(number) {
19     // Get the ASCII code for 'A'
20     a = 'A'.charCodeAt(0);
21
22     // Add number to ASCII of 'A' to get target letter
23     c = (number % 26) + a;
24
25     // Convert ASCII back to letter
26     return String.fromCharCode(c);
27 }
28
29 // Encryption function
30 function Encryption(plaintext, key) {
31
32     // Get the key by making the length same as plaintext
33     key = key.substring(0, plaintext.length);
34
35     // Empty string to store the cipher text
36     let cipher = '';
37
38     // Loop through each letter in the plaintext
39     for (let i = 0; i < plaintext.length; i++) {
40
41         // Convert letter to number, Add plaintext and key numbers, then mod 26 to get the remainder.
42         // Convert result back to letter and add to cipher
43         cipher += number_to_letter((letter_to_number(plaintext[i]) + letter_to_number(key[i])) % 26);
44     }
45
46     // Final cipher text
47     return cipher;
48 }
49
```

```

50 // Prompt function
51 function prompt() {
52     // Provide a user interface for writing output and reading input
53     const rl = readline.createInterface({
54         input: process.stdin, // input from keyboard
55         output: process.stdout // output on screen
56     });
57
58     // Question to ask the user for input
59     rl.question('Enter the plaintext: ', (plaintext) => {
60
61         // If the user key in lowercase, number of symbols
62         if (!/^[A-Z]+$/.test(plaintext)) {
63             // Error message for user
64             console.log("Error: No lowercase, numbers, or symbols allowed.\n");
65
66             // Close after printing the console log statement to ensure the output is not doubled or tripped
67             rl.close();
68
69             // Repeat the function to prompt the user again
70             prompt();
71
72             return;
73         }
74
75         // Nested function to ask the key again if it is invalid
76         function Key_again() {
77             // Question to ask the user to input the key
78             // THISISANEXAMPLEKEYINCOMPUTERSECURITYEXAM
79             rl.question('Enter the key (uppercase letters only): ', (key) => {
80                 // If the user key in lowercase, number or symbols
81                 if (!/^[A-Z]+$/.test(key)) {
82                     // Error message for user
83                     console.log("Error: No lowercase, numbers, or symbols allowed in key.\n");
84                     // Recursive function to ask the user to input the key again until the valid one appears
85                     Key_again();
86                     return;
87                 }
88
89                 // Encrypt the user input
90                 const ciphertext = Encryption(plaintext, key);
91
92                 // Output console log statements
93                 console.log("-----")
94                 console.log("          Output")
95                 console.log("-----")
96                 console.log("Plaintext:  ", plaintext); // Plaintext
97                 console.log("Key:       ", key.substring(0, plaintext.length)); // Key
98                 console.log("Cipher text: ", ciphertext); // cipher text
99                 console.log("\n")
100
101                 // Detailed output console log statements
102                 console.log("-----")
103                 console.log("          Detailed Output")
104                 console.log("-----")
105                 // Print letter with number for plain text
106                 console.log("Plaintext:  ", plaintext.split('').map(c => `${c} (${letter_to_number(c)})`).join(' '));
107                 // Print letter with number for key
108                 console.log("Key:       ", key.substring(0, plaintext.length).split('').map(c => `${c} (${letter_to_number(c)})`).join(' '));
109                 // Print letter with number for cipher text
110                 console.log("Cipher text: ", ciphertext.split('').map(c => `${c} (${letter_to_number(c)})`).join(' '));
111
112                 rl.close();
113             });
114         }
115         Key_again();
116     });
117 }
118
119 // Prompt function for the user
120 prompt();
121

```

b) Explain how one can decrypt the encrypted message using the encryption algorithm in part (a)?

We reverse the encryption process to decrypt the encrypted message. Decryption uses modular subtraction because the encryption used modular addition to combine the plaintext and key letters.

By deducting the ASCII code for "A", each letter in the ciphertext and the key is first converted to its numeric value, ranging from 0 to 25. Next, we deduct the number value of the key from the numeric value of the ciphertext to determine the original plaintext letter. If the result is negative, we add 26 to keep it within the range of 0 to 25. After that, this difference is added to the ASCII code for "A" to transform it back into a letter.

Formula to get the letter of the plaintext:

$$\text{Value of plain text} = (\text{Value of the cipher letter} - \text{Value of the key letter} + 26) \bmod 26$$

This modular subtraction allows us to get back the original plaintext for every pair of characters in the ciphertext and the key. The key must be at least as long as the ciphertext and be the same one used for encryption.

Code Screenshots:

Code Snippets

This snippet imports the readline module, which enables the program to read user input from the terminal.

```
// To take the user input import the readline module
const readline = require('readline');
```

A capital letter (A–Z) can be changed to a number between 0 and 25 using this function. For example, A turns into 0, B into 1,..., Z into 25.

```
// Assign letter to specific number
function letter_to_number(letter) {

  // Get the ASCII for A
  a = 'A'.charCodeAt(0);

  // Get the ASCII for the input letter
  b = letter.charCodeAt(0);

  // Calculate the difference to get 0 to 25 range
  calc = b - a
  return calc;
}
```

A number between 0 and 25 is converted back to a capital letter (A–Z) by this function. For example, 0 turns into A, 1 into B, and 25 into Z.

```
// Convert number (0 to 25) back to uppercase letter
function number_to_letter(number) {
  // Get the ASCII code for 'A'
  a = 'A'.charCodeAt(0);

  // Add number to ASCII of 'A' to get target letter
  c = (number % 26) + a;

  // Convert ASCII back to letter
  return String.fromCharCode(c);
}
```

The decrypt function uses modulo 26 to account for wraparound and subtracts the letter value of the key from each letter of the ciphertext to return it to its original plaintext. The decrypted message is created by processing each character individually.

```
function decrypt(ciphertext, key) {
  // Get the key by making the length same as ciphertext
  key = key.substring(0, ciphertext.length);

  // Empty string to store the plain text
  let plaintext = '';

  // Loop through each letter in the ciphertext
  for (let i = 0; i < ciphertext.length; i++) {
    // Convert cipher letter and key letter to numbers, subtract key from cipher, add 26 to avoid negative values, then mod 26
    plaintext += number_to_letter((letter_to_number(ciphertext[i]) - letter_to_number(key[i]) + 26) % 26);
  }

  return plaintext;
}
```

The prompt function asks the user for a ciphertext input, verifies that it only consists of capital letters, and then displays the original plaintext message using the decrypt function.

```

// Prompt function
function prompt() {
  // Provide a user interface for writing output and reading input
  const rl = readline.createInterface({
    input: process.stdin, // input from keyboard
    output: process.stdout // output on screen
  });

  // Question to ask the user for input
  rl.question('Enter the plaintext: ', (plaintext) => {

    // If the user key in lowercase, number of symbols
    if (!/^[A-Z]+$/.test(plaintext)) {
      // Error message for user
      console.log("Error: No lowercase, numbers, or symbols allowed.\n");

      // Close after printing the console log statement to ensure the output is not doubled or trippled
      rl.close();

      // Repeat the function to prompt the user again
      prompt();

      return;
    }

    // Nested function to ask the key again if it is invalid
    function Key_again() {
      // Question to ask the user to input the key
      // THISISANEXAMPLEKEYINCOMPUTERSECURITYEXAM
      rl.question('Enter the key (uppercase letters only): ', (key) => {
        // If the user key in lowercase, number or symbols
        if (!/^[A-Z]+$/.test(key)) {
          // Error message for user
          console.log("Error: No lowercase, numbers, or symbols allowed in key.\n");
          // Recursive function to ask the user to input the key again until the valid one appears
          Key_again();
          return;
        }

        // Encrypt the user input
        const ciphertext = Encryption(plaintext, key);

        // Output Console log statements
        console.log("-----")
        console.log("          Output")
        console.log("-----")
        console.log("Plaintext:  ", plaintext); // Plaintext
        console.log("Key:       ", key.substring(0, plaintext.length)); // Key
        console.log("Cipher text: ", ciphertext); // cipher text
        console.log("\n")

        // Detailed output console log statements
        console.log("-----")
        console.log("          Detailed Output")
        console.log("-----")
        // Print letter with number for plain text
        console.log("Plaintext:  ", plaintext.split('').map(c => `${c} (${letter_to_number(c)})`).join(' '));
        // Print letter with number for key
        console.log("Key:       ", key.substring(0, plaintext.length).split('').map(c => `${c} (${letter_to_number(c)})`).join(' '));
        // Print letter with number for cipher text
        console.log("Cipher text: ", ciphertext.split('').map(c => `${c} (${letter_to_number(c)})`).join(' '));

        rl.close();
      });
    }

    Key_again();
  });
}

```

Input/Output

Final Output:

```

Enter the ciphertext: LBJSBZRN
Enter the key (uppercase letters only): THISISANEXAMPLEKEYINCOMPUTERSECURITYEXAM
-----
Decryption
-----
Cipher text: LBJSBZRN
Key       : THISISAN
Plain text : SUBATHRA

```

Error message if the user key in lowercase or numbers or symbols in the ciphertext:

```
Enter the ciphertext: Lbjsbzn12
Error: No lowercase, numbers, or symbols allowed.
```

Error message if the user key is lowercase or numbers or symbols in the key:

```
Enter the key (uppercase letters only): Thisisexample
Error: No lowercase, numbers, or symbols allowed in key.

Enter the key (uppercase letters only): █
```

Final Code

```
1  // To take the user input import the readline module
2  const readline = require('readline');
3
4  // Assign letter to specific number
5  function letter_to_number(letter) {
6
7      // Get the ASCII for A
8      a = 'A'.charCodeAt(0);
9
10     // Get the ASCII for the input letter
11     b = letter.charCodeAt(0);
12
13     // Calculate the difference to get 0 to 25 range
14     calc = b - a;
15     return calc;
16 }
17
18 // Convert number (0 to 25) back to uppercase letter
19 function number_to_letter(number) {
20     // Get the ASCII code for 'A'
21     a = 'A'.charCodeAt(0);
22
23     // Add number to ASCII of 'A' to get target letter
24     c = (number % 26) + a;
25
26     // Convert ASCII back to letter
27     return String.fromCharCode(c);
28 }
29
30 function decrypt(ciphertext, key) {
31     // Get the key by making the length same as ciphertext
32     key = key.substring(0, ciphertext.length);
33
34     // Empty string to store the plain text
35     let plaintext = '';
36
37     // Loop through each letter in the ciphertext
38     for (let i = 0; i < ciphertext.length; i++) {
39
40         // Convert cipher letter and key letter to numbers, subtract key from cipher, add 26 to avoid negative values, then a
41         plaintext += number_to_letter((letter_to_number(ciphertext[i]) - letter_to_number(key[i]) + 26) % 26);
42     }
43
44     return plaintext;
45 }
46
```

```

47 function prompt() {
48   // Provide a user interface for writing output and reading input
49   const rl = readline.createInterface({
50     input: process.stdin, // input from keyboard
51     output: process.stdout // output on screen
52   });
53
54   // Question to ask the user for input
55   rl.question('Enter the ciphertext: ', (ciphertext) => {
56
57     // If the user key in lowercase, number or symbols
58     if (/^[A-Z]+$/.test(ciphertext)) {
59       // Error message for user
60       console.log("Error: No lowercase, numbers, or symbols allowed.\n");
61
62       // Close after printing the console log statement to ensure the output is not doubled or trippled
63       rl.close();
64
65       // Repeat the function to prompt the user again
66       prompt();
67       return;
68     }
69
70
71     // Nested function to ask the key again if it is invalid
72     function Key_again() {
73       // Question to ask the user to input the key
74       // "THISISANEXAMPLEKEYINCOMPUTERSECURITYEXAM"
75       rl.question("Enter the key (uppercase letters only): ", (key) => {
76         // If the user key in lowercase, number or symbols
77         if (/^[A-Z]+$/.test(key)) {
78           // Error message for user
79           console.log("Error: No lowercase, numbers, or symbols allowed in key.\n");
80           // Recursive function to ask the user to input the key again until the valid one appears
81           Key_again();
82           return;
83         }
84
85         // Decrypt the input
86         const plaintext = decrypt(ciphertext, key);
87
88         // Output Console log statements
89         console.log("-----")
90         console.log("          Decryption")
91         console.log("-----")
92         console.log("Cipher text:", ciphertext); // ciphertext
93         console.log("Key       :", key.substring(0, ciphertext.length)); // Key
94         console.log("Plain text :", plaintext); // plaintext
95
96         // Close the prompt
97         rl.close();
98       });
99     }
100    Key_again();
101  });
102 }
103
104 // Prompt function for the user
105 prompt();

```

- c) Under what conditions (specify two) can a cipher be broken, meaning that an attacker can decipher the encrypted message without having the key? Support your reasoning with example(s)

Condition 1:

Without the key, a cipher can be cracked if the attacker has access to selected plaintext pairs or known plaintext. The attacker is aware of certain parts of the original message and its encrypted version in a known plaintext attack. The key or encryption technique can be found by the attacker through analysing these pairs and finding patterns or mathematical relationships. For example, in a basic Caesar cipher, an attacker can

quickly determine the shift value and decrypt the entire message if they are familiar with a few plaintext letters and their ciphertext equivalents. Assume that the Caesar cipher was used to encrypt the original message, which contained the name "Subathra," and that the result is "Vxedwkwd." An attacker can determine that each letter is shifted by +3 positions by comparing 'S' to 'V', u to x, b to e and so on. The attacker can decrypt the entire message by using this name alone to determine that the Caesar cipher employed a shift of +3. This is made possible by the Caesar cipher's limited keyspace, which allows for simple pattern-based or brute-force analysis with only 25 possible shifts.

Condition 2:

If the encryption key is too short or is used repeatedly, another situation arises. Predictable patterns appear in the ciphertext if a one-time pad key is mistakenly reused or if a key is shorter than the message and is repeated, as in the Vigenère cipher. The key can be recovered and the message decrypted by taking advantage of these patterns through statistical or frequency analysis. For example, during World War II, cryptanalysts were able to decipher Soviet encrypted messages by using reused one-time pad keys in the Venona Project.

d) Identify THREE key factors for defining a robust key to enhance algorithm security? Explain your reasons

1) Factor 1: Key Length

The length of a strong encryption key is necessary to withstand brute-force attacks, in which a hacker systematically attempts every key combination. Because there are more combinations with longer keys, breaking them takes exponentially more time and computing power. By offering 2^{256} possible combinations, a 256-bit key used in AES encryption, for instance, makes it nearly impossible for even the most powerful computer to guess. Therefore, the encryption algorithm's security is greatly increased by selecting a key size that is too large.

2) Factor 2: Randomness

For the key to be truly unpredictable, it should be generated using a cryptographically secure random number generator. Dictionary or social engineering attacks are easy ways to guess keys based on patterns, repeated sequences, or personal information, such as names or birthdays. Randomness gets rid of predictable structures and makes it harder for an attacker to figure out or copy the key. For the encryption to stay robust even when examined, this randomness is necessary.

3) Factor 3: Key Confidentiality and Proper Management

If a key is not properly managed or kept safe, even the most complicated and random one can become vulnerable. Avoiding techniques like hardcoding keys into source code and keeping keys in safe places are just a few of the secure key management procedures that are necessary to stop unwanted access. Keeping the keys safe throughout their lifecycle is just as important to encryption security as the algorithm. A key leak or exposure affects the encryption system as a whole.

Question 2

- a) **What is the purpose of padding in cryptography algorithms such as DES? Explain it using a simple example**

DES encrypts data in fixed-size blocks instead of as a continuous stream because it is a block cipher. Cryptographic algorithms such as DES use padding to make sure that the input data exactly fits into blocks of a certain size. Although DES used 64-bit (8-byte) blocks, messages are frequently not exactly divisible by this size. To make the final block the necessary length in these situations, additional padding bits are added. Without padding, DES would not know how to handle short blocks, and decryption could result in inaccurate or distorted data. Encryption and decryption are guaranteed to be reliable and secure with padding.

Example:

Message to encrypt: HELLO

Each character = 1 byte, so HELLO = 5 bytes

But the DES requires 8 bytes per block. So, we need to add 3 more bytes of padding to complete the block. A common method is to repeat the number of padding bytes.

Original:	H	E	L	L	O			
Padding:	03	03	03					
Final block:	H	E	L	L	O	03	03	03
Final block (hex):	48	45	4C	4C	4F	03	03	03

During decryption, DES will read the last byte 03, meaning 3 padding bytes and remove them, restoring the original message HELLO.

- b) **We know that the size of the key in DES is 64 bits, assume for padding we just add 0 at the end of the plaintext (the original message), what is the result of padding for the following text? At first replace the “YYY” with the first 3 letters of your name or family then calculate the padding (show the result in hexadecimal format): “YYYComputerSecurity” Show your answer step by step.**

Step 1: Replace the YYY to get the message to encrypt

Message to encrypt: SUBComputerSecurity

Step 2: Count the number of bytes

Each character = 1 byte

S U B C o m p u t e r S e c u r i t y

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
So, the message has 19 bytes.

Step 3: Find how many bytes are needed to complete the block

DES runs in block of 8 bytes.

So, break the message into 8-byte separately:

1st block: SUBCompu (8 byte)

2nd block: terSecur (8 byte)

3rd block: ity (3 byte)

To make the final block eight bytes, we must add five more bytes because the 3rd block is only three.

Step 4: Apply padding

Add 0 for the 3rd block to make it 8 byte.

3rd block: ity00000 (8 byte)

Original message: S U B C o m p u t e r S e c u r i t y

Padding: 0 0 0 0 0

Final block: S U B C o m p u t e r S e c u r i t y 0 0 0 0 0

Step 5: Convert each letter to ASCII values

Character	ASCII
S	83
U	85
B	66
C	67
o	111
m	109
p	112
u	117
t	116
e	101
r	114
S	83
e	101
c	99
u	117
r	114
i	105
t	116
y	121
0	48
0	48
0	48
0	48
0	48

Step 5: Based on the ASCII Value, using the conversion table, convert each block message to Hexadecimal

Block 1: S U B C o m p u
 ASCII Value: 83 85 66 67 111 109 112 117
 Hex value: 53 55 42 43 6F 6D 70 75

Block 2: t e r S e c u r
 ASCII Value: 116 101 114 83 101 99 117 114
 Hex value: 74 65 72 53 65 63 75 72

Block 3: i t y 0 0 0 0 0
 ASCII Value: 105 116 121 48 48 48 48 48
 Hex value: 69 74 79 30 30 30 30 30

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	33	21	!	65	41	A	97	61	a
1	1	[START OF HEADING]	34	22	"	66	42	B	98	62	b
2	2	[START OF TEXT]	35	23	#	67	43	C	99	63	c
3	3	[END OF TEXT]	36	24	\$	68	44	D	100	64	d
4	4	[END OF TRANSMISSION]	37	25	%	69	45	E	101	65	e
5	5	[ENQUIRY]	38	26	&	70	46	F	102	66	f
6	6	[ACKNOWLEDGE]	39	27	'	71	47	G	103	67	g
7	7	[BELL]	40	28	(	72	48	H	104	68	h
8	8	[BACKSPACE]	41	29	)	73	49	I	105	69	i
9	9	[HORIZONTAL TAB]	42	2A	*	74	4A	J	106	6A	j
10	A	[LINE FEED]	43	2B	+	75	4B	K	107	6B	k
11	B	[VERTICAL TAB]	44	2C	,	76	4C	L	108	6C	l
12	C	[FORM FEED]	45	2D	-	77	4D	M	109	6D	m
13	D	[CARRIAGE RETURN]	46	2E	.	78	4E	N	110	6E	n
14	E	[SHIFT OUT]	47	2F	/	79	4F	O	111	6F	o
15	F	[SHIFT IN]	48	30	0	80	50	P	112	70	p
16	10	[DATA LINK ESCAPE]	49	31	1	81	51	Q	113	71	q
17	11	[DEVICE CONTROL 1]	50	32	2	82	52	R	114	72	r
18	12	[DEVICE CONTROL 2]	51	33	3	83	53	S	115	73	s
19	13	[DEVICE CONTROL 3]	52	34	4	84	54	T	116	74	t
20	14	[DEVICE CONTROL 4]	53	35	5	85	55	U	117	75	u
21	15	[NEGATIVE ACKNOWLEDGE]	54	36	6	86	56	V	118	76	v
22	16	[SYNCHRONOUS IDLE]	55	37	7	87	57	W	119	77	w
23	17	[END OF TRANS. BLOCK]	56	38	8	88	58	X	120	78	x
24	18	[CANCEL]	57	39	9	89	59	Y	121	79	y
25	19	[END OF MEDIUM]	58	3A	:	90	5A	Z	122	7A	z
26	1A	[SUBSTITUTE]	59	3B	;	91	5B	[	123	7B	{
27	1B	[ESCAPE]	60	3C	<	92	5C	\	124	7C	
28	1C	[FILE SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
29	1D	[GROUP SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
30	1E	[RECORD SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]
31	1F	[UNIT SEPARATOR]									

Img 1: ASCII to Hexadecimal conversion table

Final Output:

535542436F6D707574657253656375726974793030303030

- c) We know that RSA works with numbers. Assume we want to encrypt the plaintext $M = \text{"ComputerSecurity"}$. We know we should use the following formula: $C = M^e \bmod N$ in which (e, N) is the public key. But before using this formula, at first, we should convert M into a number. To do so, there are different solutions. Search the Internet and explain one of them. Show different steps of the selected solution on the given plaintext $M = \text{"ComputerSecurity"}$ as an example. Consider padding as well.

Since RSA only works with integers, we must first convert a plaintext message into a number before we can use it to encrypt it. The public key is (e, N), and the encryption formula is $C = M^e \bmod N$. To guarantee security, we must pad the message before encryption, and RSA can only encrypt messages that are numerical and smaller than the modulus N. According to my online research, using UTF-8 encoding followed by OAEP (Optimal Asymmetric Encryption Padding) is a safe and standardized method that combines padding and message to number conversion.

First, we use UTF-8 encoding to convert the plaintext into a number. Each character is converted into one or more bytes using the UTF-8 character encoding standard. It is widely used in secure systems and supports all Unicode characters. Since all the characters in the string "ComputerSecurity" are standard ASCII, each character will be represented by a one-byte value when encoded in UTF-8.

UTF-8 Conversion:

Character	UTF-8 (Hex)
C	0x43
o	0x6F
m	0X6D
p	0X70
u	0X75
t	0X74
e	0X65
r	0X72
S	0X53
e	0X65
c	0X63
u	0X75
r	0X72
i	0X69
t	0X74
y	0X79

Combining these byte values, we get the full UTF-8 byte:

43 6f 6d 70 75 74 65 72 53 65 63 75 72 69 74 79

It is now possible to interpret this byte stream as a large integer. For instance, we can obtain a number appropriate for RSA encryption by combining the hexadecimal values into a single hex number, **436f6d70757465725365637572697479**, and then converting it to decimal, as long as it is less than the modulus N. However, since RSA is predictable and patterns are readily identifiable, just converting the message into a number and encrypting it is not secure. OAEP padding can help with that.

To stop attackers from identifying patterns in ciphertext, OAEP adds structure and randomness. For example, OAEP first pads the plain the byte values of C of (0x43) and o of (0x6f) with random data and hashes before directly encrypting them.

Suppose C becomes a padded 128-bit block like:

0xB1A2C3D4E5F60718293A4B5C6D7E8F90

which converts to the decimal number:

236519772638168855050605940623220489232

While o might result in completely different padded block like:

0x9F8E7D6C5B4A39281706F5E4D3C2B1A0

which converts to the decimal number:

212698848468099774274927948725684709856

These padded values are then encrypted with RSA using $C = M^e \bmod N$, where (e, N) is the public key, ensuring even identical characters never produce the same ciphertext. The ciphertext C is the encrypted output sent securely over the network.

d) What is a digital signature? Is it possible to generate a digital signature using RSA? What about DES? Explain your reasoning

A digital signature is a cryptographic method that confirms the integrity and credibility of a digital document or message. Although it is far more secure, it functions similarly to a handwritten signature or a stamped seal. The message's creation by a known sender and its integrity during transmission are verified by a digital signature.

RSA is an asymmetric encryption algorithm, which means it uses two keys, a public key for verification and a private key for signing, so it can be used to create a digital signature. When using RSA digital signatures, the sender "signs", a hash of the message with their private key, and the recipient verifies it using the sender's public key. Because the private key generated signature can only be verified by the matching public key, this works.

Digital signatures, however, cannot be created using DES as it is a symmetric encryption algorithm. The same key is used for both encryption and decryption when using the symmetric encryption algorithm. Since the secret key is shared by the sender and the recipient, it is impossible to determine who wrote the message. DES is therefore unable to differentiate between the signer and the verifier, which prevents it from offering the assurance of confidentiality necessary for digital signatures. It does not have the key separation required for safe, authenticated signing.

References

GeeksforGeeks. *ASCII Table - Characters and their codes*. [online] Available at: <https://www.geeksforgeeks.org/dsa/ascii-table/> [Accessed 7 Jun 2025]

Pointcheval, D. (2011). *OAEP: Optimal Asymmetric Encryption Padding*. [online] Available at: https://link.springer.com/rwe/10.1007/978-1-4419-5906-5_150 [Accessed 7 Jun 2025]

Zhong, Y. (2022). An Overview of RSA and OAEP Padding. [online] Available at: <https://drpress.org/ojs/index.php/HSET/article/view/431> [Accessed 7 Jun 2025]

Juviler, J. (n.d.). *What is UTF-8 Encoding? A Guide for Non-Programmers*. [online] Available at: <https://blog.hubspot.com/website/what-is-utf-8> [Accessed 7 Jun 2025]